PYTHON OPTIMIZATION

Кешування в Python

Мистецтво пам'ятати лише потрібне

Теорія та практика мемоізації, політик видалення та архітектурних патернів

Сумець Світлана

2026

Навіщо потрібне кешування?

> Кешування - це стратегічний підхід до оптимізації, що дозволяє системі "пам'ятати" минулі зусилля заради майбутньої швидкості.

Зменшення повторних обчислень

Уникнення дорогих операцій CPU для ідентичних вхідних даних. "Не рахуй двічі те, що вже порахував".

Зниження латентності

Миттєва віддача даних з пам'яті (RAM) замість повільного читання з диску або мережевих запитів.

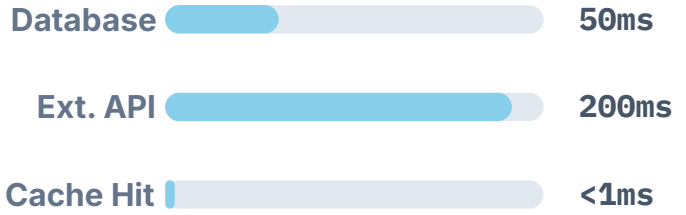
Стабільна продуктивність

Згладжування пікових навантажень (traffic spikes) та захист бази даних від перевантаження.

Економія ресурсів

Зменшення витрат на інфраструктуру (менше серверів БД, менший мережевий трафік).

ВПЛИВ НА ШВИДКІСТЬ (МС)



Ключовий інсайт: Кеш перетворює "дорогі" операції I/O на "дешеві" операції читання з пам'яті, прискорюючи систему.

Основи: мемоізація та детермінізм

> Кешування ефективне лише тоді, коли результат функції залежить виключно від її аргументів.

Мемоізація (Memoization)

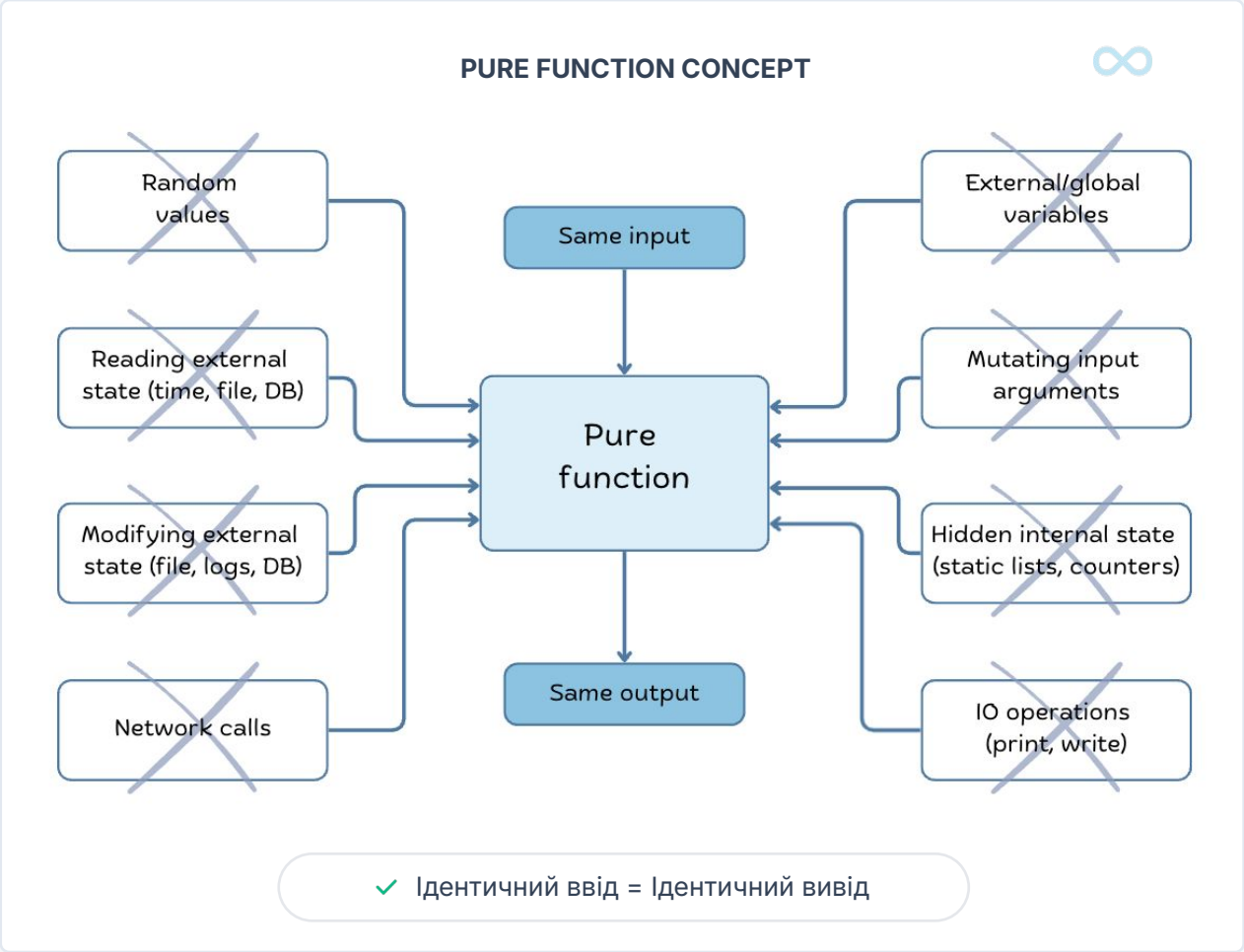
Стратегія "зверху-вниз": збереження результату виконання функції для уникнення повторних обчислень при тих самих вхідних даних.

Табуляція (Tabulation)

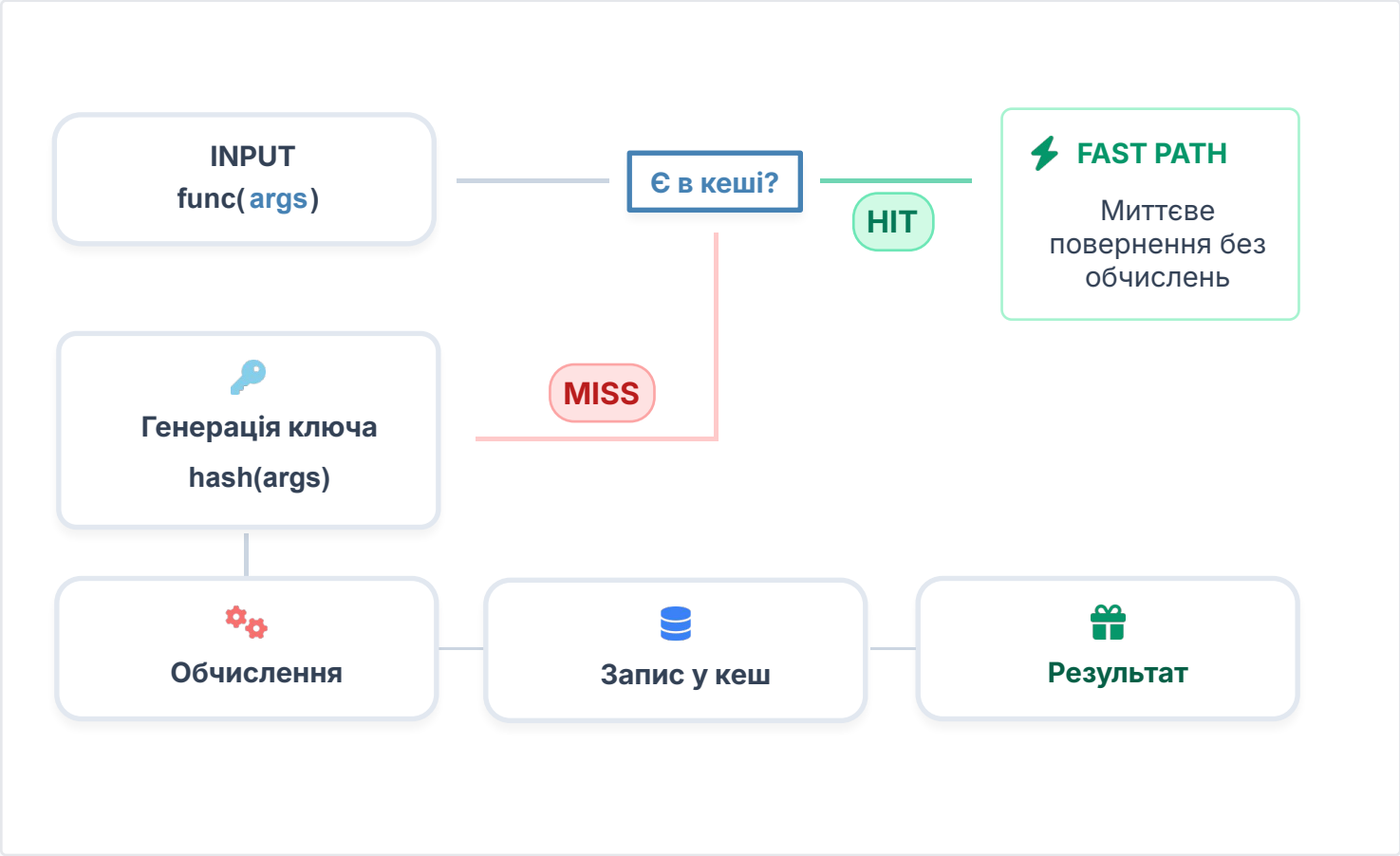
Підхід "знизу-вверх": ітеративне заповнення таблиці результатів від найпростіших підзадач до основної мети.

Детермінізм

f(x) завжди повертає y. Чисті функції не мають побічних ефектів (I/O, random, global state), тому їх безпечно кешувати.



Потік роботи мемоізації



Принцип роботи

Мемоізація обгортає функцію прошарком, який перехоплює кожен виклик.

Ключ формується унікально для кожної комбінації аргументів.

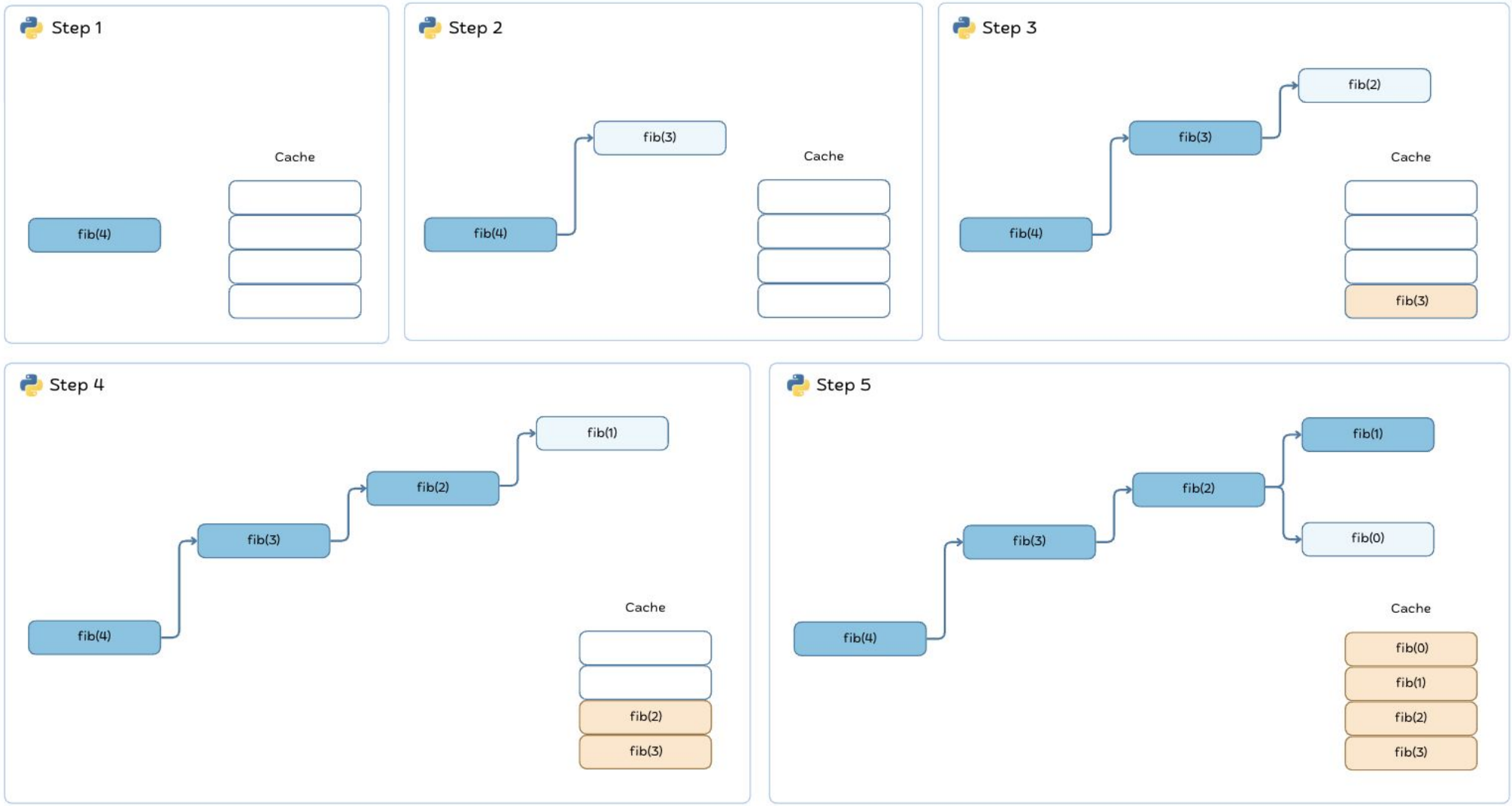
Якщо ключ знайдено (Hit), обчислення пропускаються.

HIT VS MISS

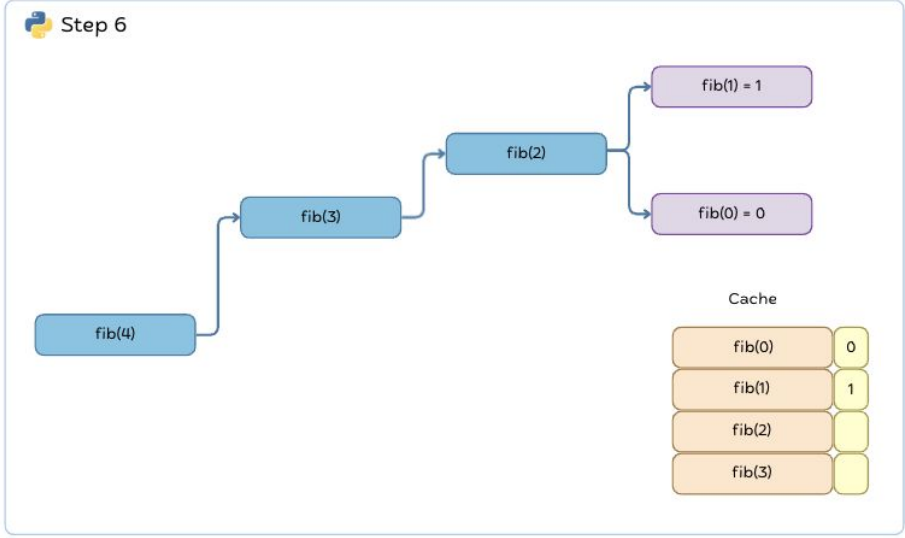
✓	CACHE HIT O(1) Access	~0.1μs
✕	CACHE MISS O(N) Compute	>100ms

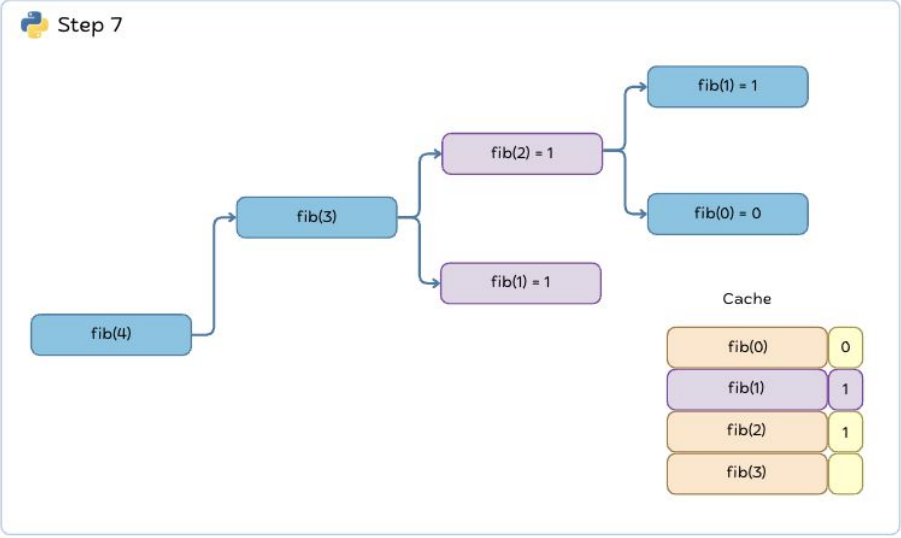
** Час залежить від складності функції*

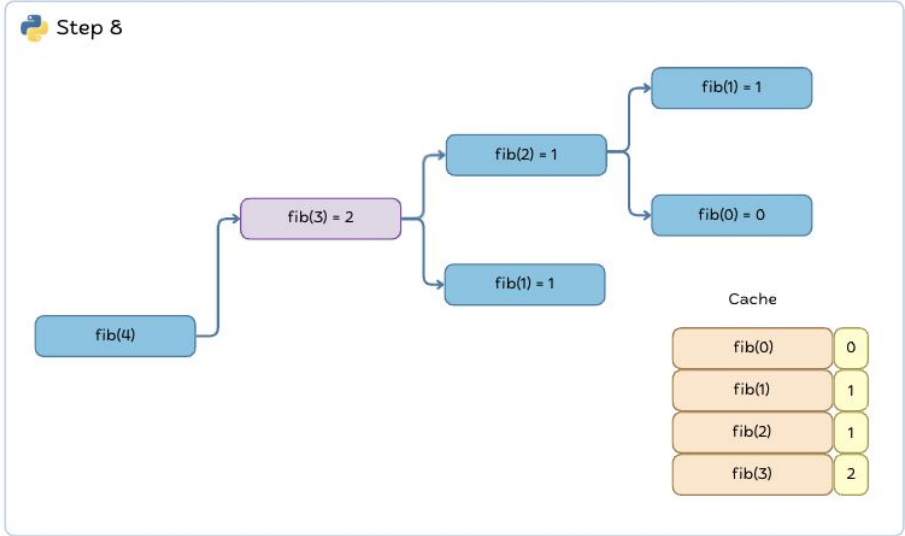
Фібоначчі з кешем (мемоізація)

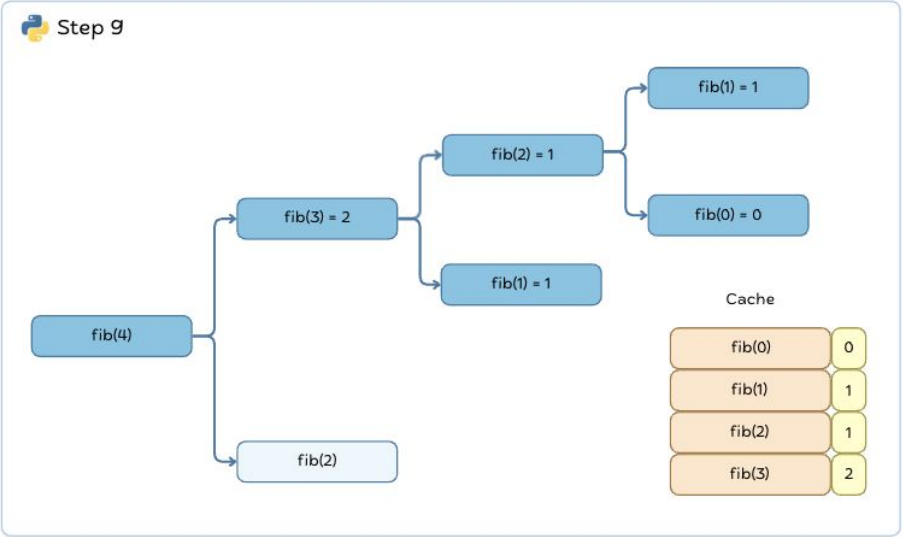


Фібоначчі з кешем (мемоізація)

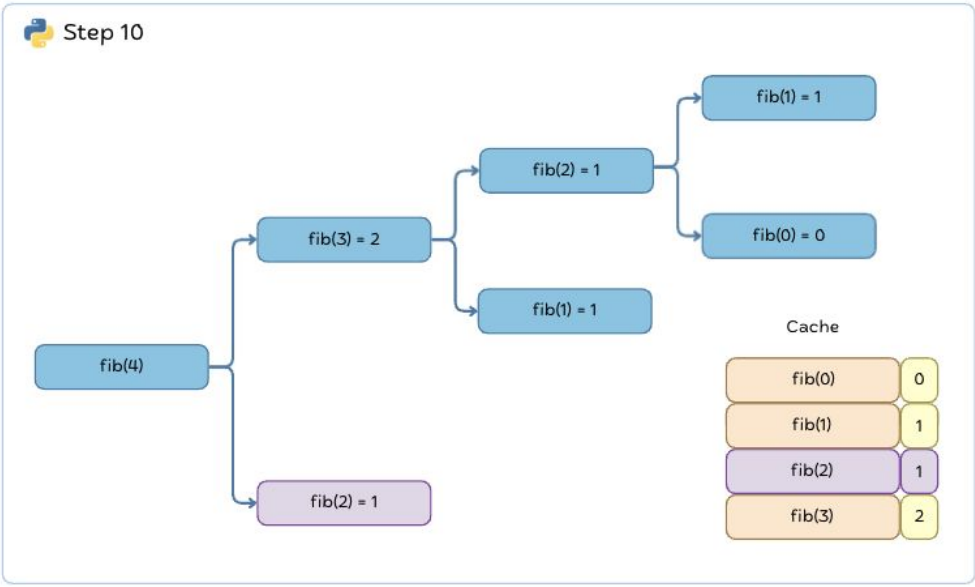


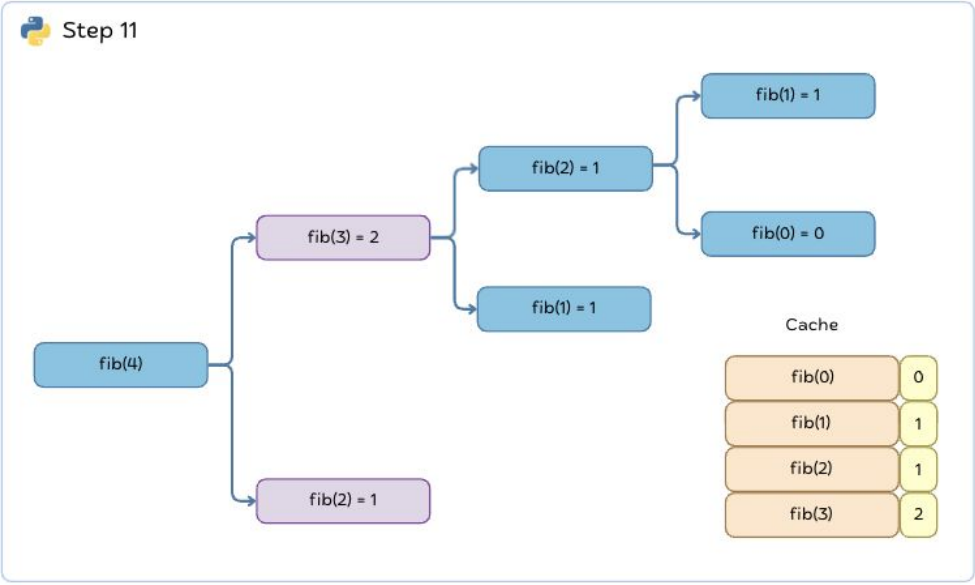






Фібоначчі з кешем (мемоізація)





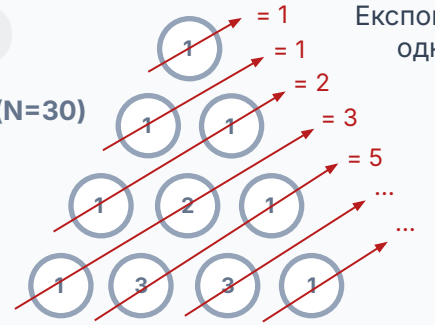
Рекурсія без кешу

Рекурсія

СКЛАДНІСТЬ: O(2^n)

КІЛЬКІСТЬ ВИКЛИКІВ (N=30)

2,692,537



Експоненціальний ріст: одні й ті ж значення обчислюються багаторазово

VS

З мемоізацією

@lru_cache

СКЛАДНІСТЬ: O(n)

КІЛЬКІСТЬ ВИКЛИКІВ (N=30)

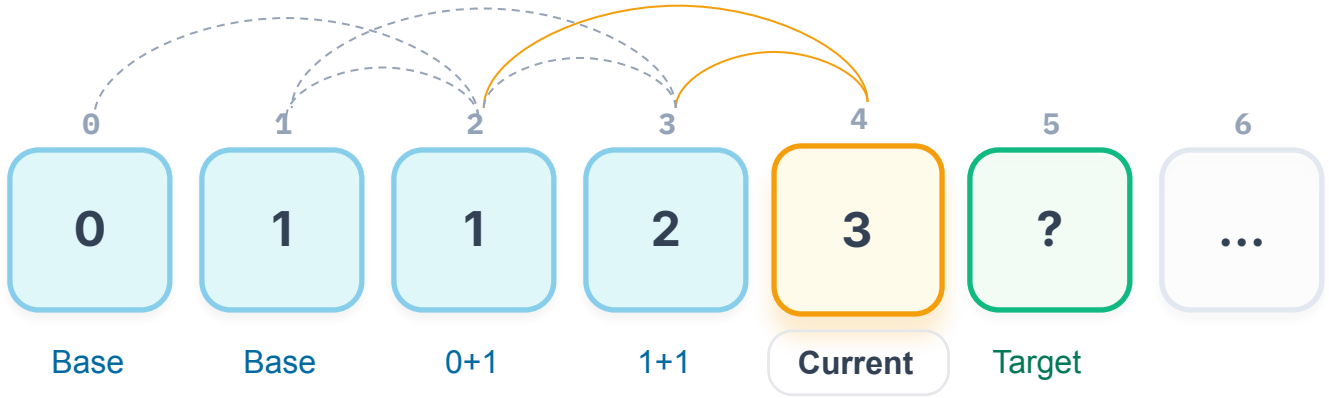
31

Лінійний прохід: кожен аргумент обчислюється лише 1 раз

1 → 2 → 3 → 4 → 5

Табуляція (Bottom-Up)


```
dp[i] = dp[i-1] + dp[i-2]
// Ітеративне заповнення таблиці
```

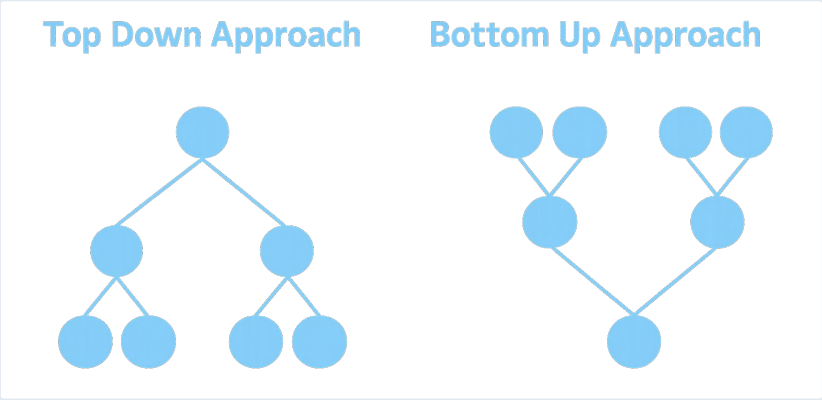


Обчислені (i < k) Поточний крок (i = k) Майбутні (i > k)

Переваги підходу

Відсутність Рекурсії Використовується простий цикл for. Немає ризику Stack Overflow Error.

- **Передбачуваність**
  Всі стани обчислюються послідовно. Пам'ять виділяється заздалегідь.
- **Оптимізація пам'яті**
  Можна зберігати лише останні 2 значення, звівши пам'ять до O(1).



Фібоначчі з кешем (табуляція)

table	0	0	0	0	0
	0	1	2	3	4

Create an arrey to store the results and initialize it with 0's

table	0	1	0	0	0
	0	1	2	3	4

Set the initial values for F(0) and F(1) in the array

table	0	1	1	0	0
	0	1	2	3	4

Start from the second index and calculate the values using the following formula: F(i) = F(i - 1) + F(i - 2)
In this case, F(2) = F(1) + F(0) = 1 + 0 = 1

table	0	1	1	2	0
	0	1	2	3	4

Iterate through the array and calculate the values
In this case, F(3) = F(2) + F(1) = 1 + 1 = 2

table	0	1	1	2	3
	0	1	2	3	4

Calculate the final output. The last index now contains the Fibonacci of F(4)
In this case, F(4) = F(3) + F(2) = 2 + 1 = 3

Мемоізація

Top-Down

↓ Рекурсивний підхід "зверху-вниз"

Стратегія	`On-Demand (Лінива)`
Реалізація	`Recursion + Cache`
Перевага	`Тільки потрібні стани`

VS

Табуляція

Bottom-Up

↑ Ітеративний підхід "знизу-вгору"

i=0 i=1 i=2 i=3 i=4 i=5

Стратегія	`Eager (Повна)`
Реалізація	`Iteration + Table`
Перевага	`Без Stack Overflow`

Політики видалення (Eviction Policies)

> Коли пам'ять заповнена, кеш повинен вирішити, кого "виселити". Вибір правильного алгоритму критичний для ефективності.

TTL (Time To Live)

Елемент живе фіксований час. Гарантує свіжість даних, але не враховує популярність.

LRU (Least Recently Used)

Видаляє ті, що найдовше не використовувались. Золотий стандарт для більшості веб-додатків.

LFU (Least Frequently Used)

Видаляє ті, що найрідше запитувались. Захищає "історично популярні" дані від вимивання новими.

FIFO (First In, First Out)

Найпростіша черга: хто перший прийшов — перший пішов. Швидка, але неефективна при повторних зверненнях.

МАТРИЦЯ ВИБОРУ

DEFAULT

Загальний трафік

Користувачі мають схожі інтереси ("останні новини").

Вибір: LRU

STABLE

Популярний контент

Деякі дані завжди популярні (головна сторінка, логотип).

Вибір: LFU

FRESH

Чутливі до часу

Курси валют, сесії користувачів, токени доступу.

Вибір: TTL

SCAN

Циклічний обхід

Обробка великих даних один раз без повторів.

Вибір: MRU / FIFO

Правило великого пальця: Починайте з LRU. Це найкращий баланс між простотою та ефективністю для 90% випадків.

Як працює LRU (Least Recently Used)

HEAD (MOST RECENT)

TAIL (LEAST RECENT)

K1 DataA — K3 DataC — K2 DataB — K4 DataD

Next to Evict

У Python реалізовано в functools.lru_cache та OrderedDict.

```
from functools import lru_cache

@lru_cache(maxsize=16)
def compute_fib_plain(n):
    . . .
```

Hash Map

"K1":	Node_Ptr → [DataA]
"K2":	Node_Ptr → [DataB]
"K3":	Node_Ptr → [DataC]
"K4":	Node_Ptr → [DataD]

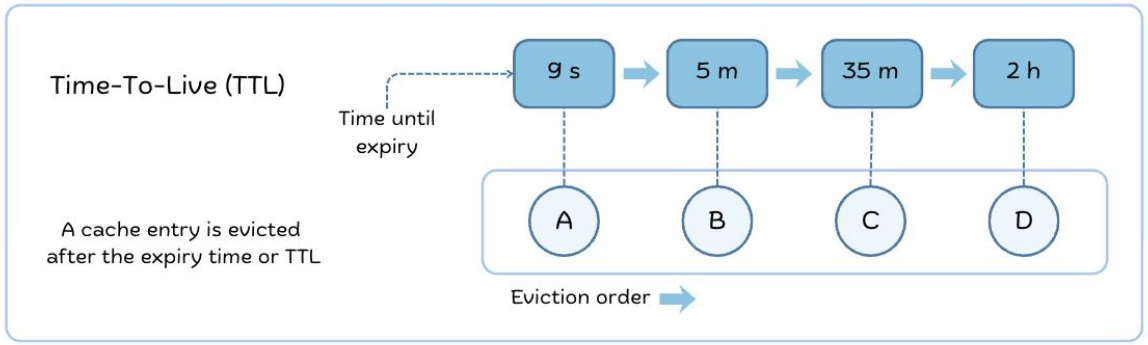
Забезпечує миттєвий доступ до будь-якої ноди

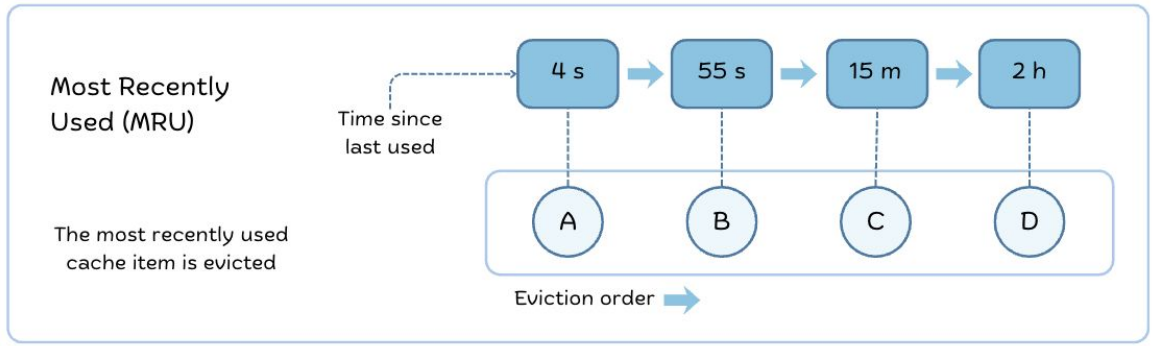
ШВИДКОДІЯ O(1)

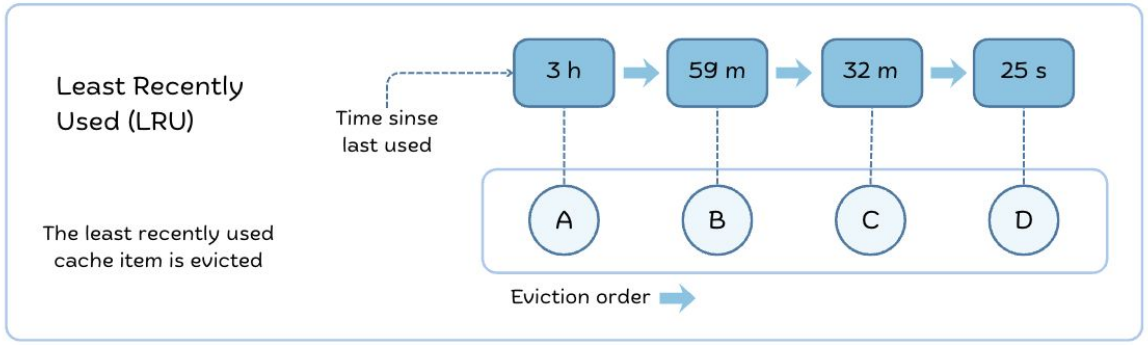
Get (Read)
Hash lookup + Move to Head — O(1)

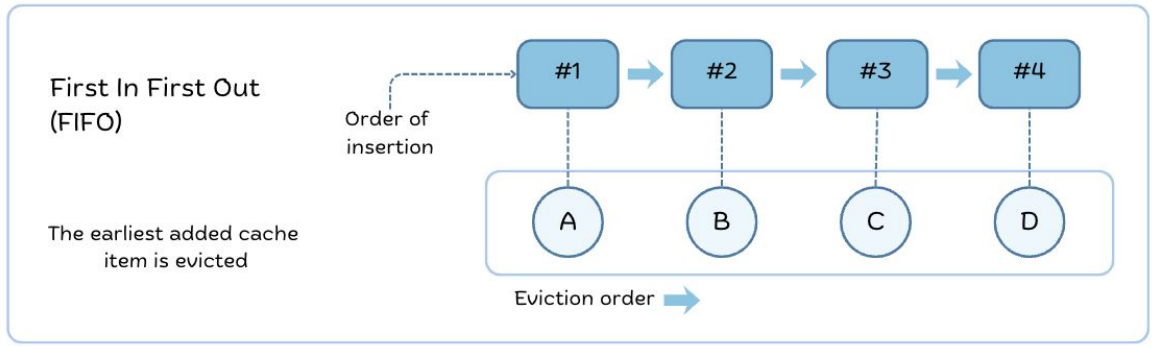
Put (Write)
Add Head (+ Evict Tail if full) — O(1)

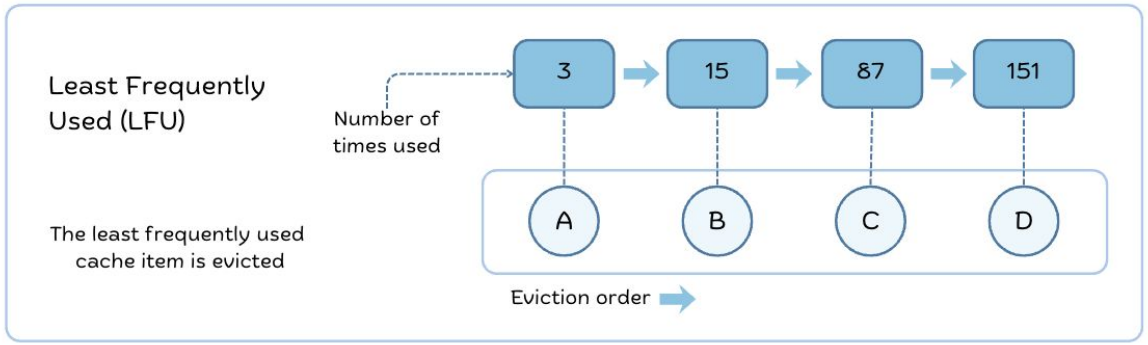
Політики кешування











- Політика визначає кандидата на видалення
- Час ≠ частота доступу
- Реальні кеші часто використовують гібридні підходи
- Усі політики застосовуються лише коли кеш переповнений

Порівняння: LRU vs LFU vs FIFO

НАЙПРОСТІШИЙ

FIFO (Queue)

Hit Ratio: ~32%

⚠ Belady's Anomaly

Overhead	Низький (O(1))
Zipf-навантаження	Погано
Адаптація до трендів	Повільна
Стійкість до сканування	Низька

Видаляє старі дані незалежно від частоти/актуальності

ЗОЛОТИЙ СТАНДАРТ

LRU (Recency)

Hit Ratio: ~45%

✓ Best General Choice

Overhead	Середній (O(1))
Zipf-навантаження	Добре
Адаптація до трендів	Швидка
Стійкість до сканування	Середня

Оптимальний баланс для більшості веб-сервісів

ЧАСТОТНИЙ АНАЛІЗ

LFU (Frequency)

Hit Ratio: ~52%

Max Hit Ratio (Zipf)

Overhead	Високий (O(log N))
Zipf-навантаження	Відмінно
Адаптація до трендів	Дуже повільна
Стійкість до сканування	Висока

Зберігає популярні, ігнорує одноразові запити

LRU - безпечний дефолт. LFU - краще для стабільної популярності (відео, товари). FIFO - лише для простих буферів.

Metrics based on standard Zipf workload

Розмір кешу

Rules for choosing size

Empirical measurements

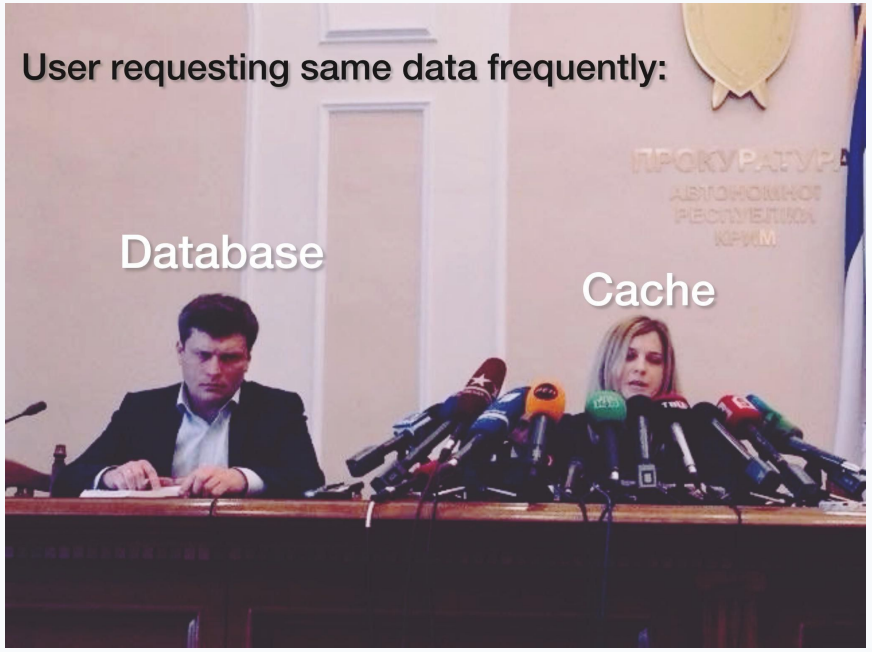
Test performance with different cache sizes to find what works best

Task characteristics

Analyze the nature of input data to determine a suitable cache size

Object size

Evaluate the size of cached objects to avoid excessive memory usage



Кеш - пресслужба бази даних

Оптимальний розмір кешу (WSS)

WSS Working Set Size

Об'єм "гарячих" даних, до яких найчастіше звертається програма. Це мінімальний набір даних, необхідний для ефективної роботи без постійних звернень до повільної пам'яті.

α Коефіцієнт запасу

Рекомендоване значення: 1.5 – 3.0. Додатковий буфер дозволяє обробляти сплески навантаження та нові дані, не витісняючи основний робочий набір.

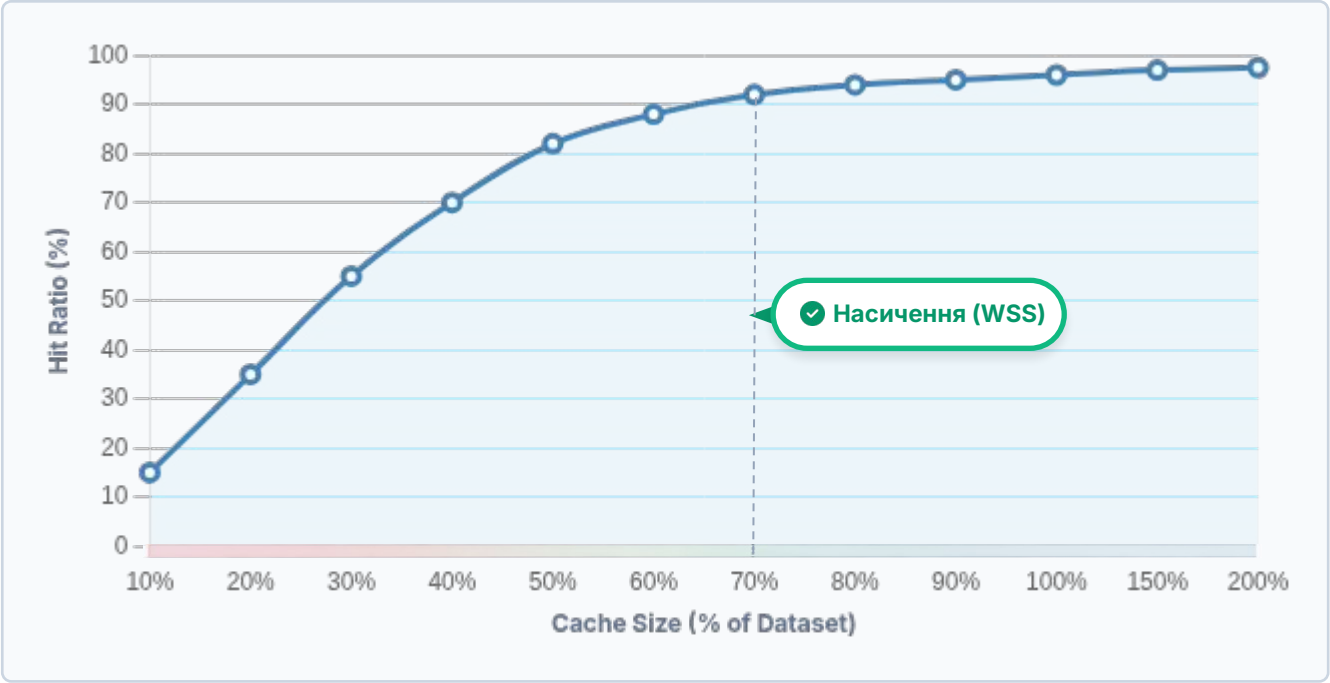
Ризики

Занадто малий кеш → постійні промахи (Thrashing).
Занадто великий кеш → марна витрата пам'яті без приросту Hit Ratio.

Емпіричне правило

$$C_{opt} = \alpha * WSS, \text{ де } \alpha \in [1.5, 3]$$

Hit Ratio Curve



THRASHING ZONE
Розмір < WSS

OPTIMAL ZONE
Розмір ≈ 1.5 × WSS

DIMINISHING RETURNS
Розмір >> WSS

Розподіл Ципфа та "Гарячі" дані

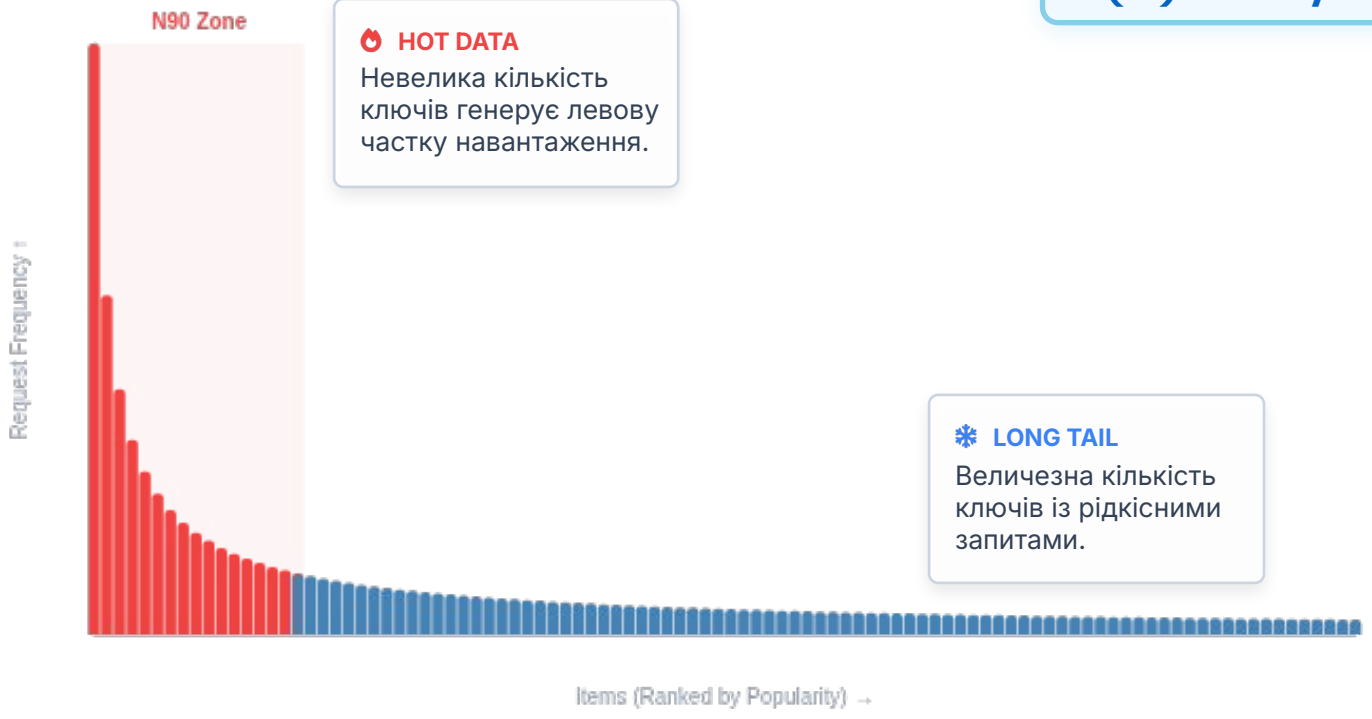
$$C_{opt} = \min \{k \mid H(k + \Delta) - H(k) < \varepsilon\}$$

$$C_{opt} \approx N_{90}$$

FREQUENCY OF ACCESS

Правило 90/10 (Power Law)

$$f(r) \propto 1/r$$



HOT DATA
Невелика кількість ключів генерує левову частку навантаження.

LONG TAIL
Величезна кількість ключів із рідкісними запитами.

Показник N90

Top 10-15%

Мінімальна кількість об'єктів у кеші, що покриває 90% усіх запитів.

Прогрів кешу

Warm-up

Попереднє завантаження N90-ключів після рестарту, щоб уникнути "холодного старту".

ВПЛИВ НА СТРАТЕГІЮ

Економія пам'яті
Немає сенсу кешувати "довгий хвіст" (Long Tail), це марна трата RAM.

LFU vs LRU
Для статичного розподілу Ципфа LFU часто ефективніший за LRU.

Архітектурні патерни кешування

> Вибір стратегії взаємодії між додатком, кешем та базою даних визначає надійність та швидкість системи.

Cache Aside (Lazy Loading)

Додаток сам керує кешем: перевіряє наявність, читає з БД при промаху та оновлює кеш. Найпопулярніший патерн.

Cache Through

Кеш є головною точкою входу. Додаток читає/пише лише в кеш, а кеш синхронно оновлює БД.

Cache Ahead

Упереджувальне завантаження даних до того, як вони знадобляться (prediction), часто для аналітики.

Write-behind

Асинхронний запис: дані пишуться в кеш і миттєво повертається відповідь, а в БД переносяться фоном.

СХЕМА ВЗАЄМОДІЇ

Cache Aside
App ↔ Cache
App ↔ DB

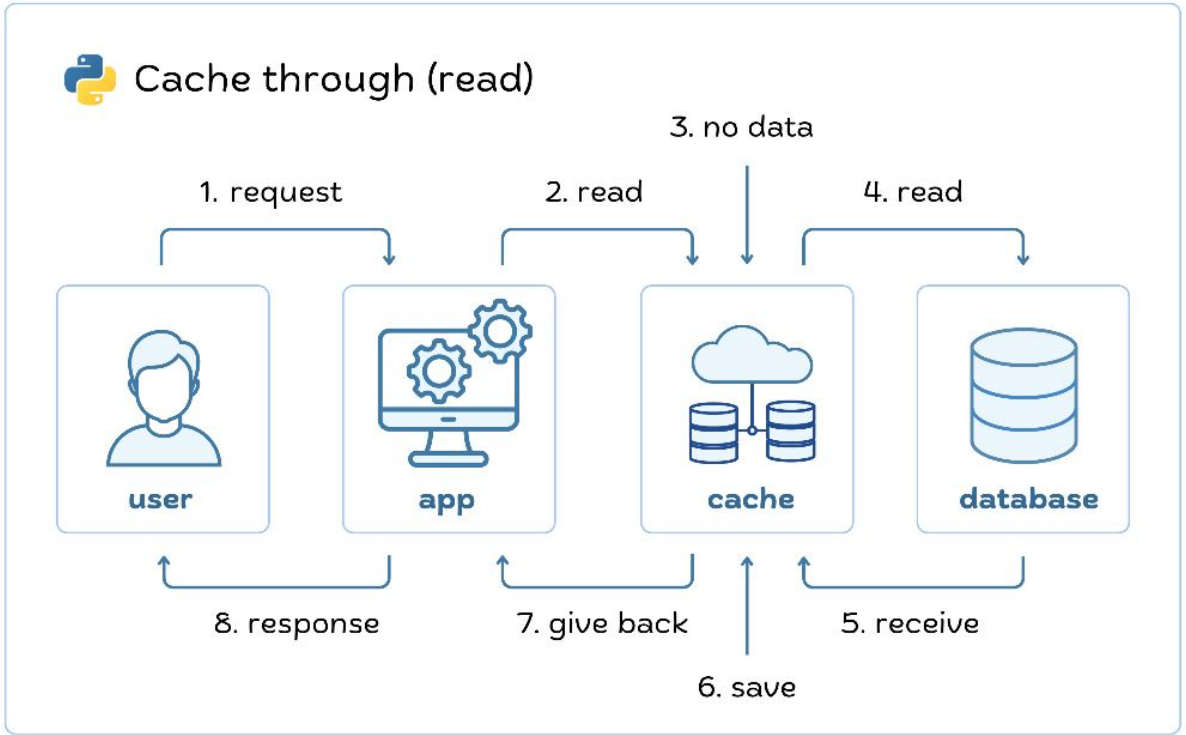
Read/Write Through
App ↔ Cache ↔ DB
(Synchronous)

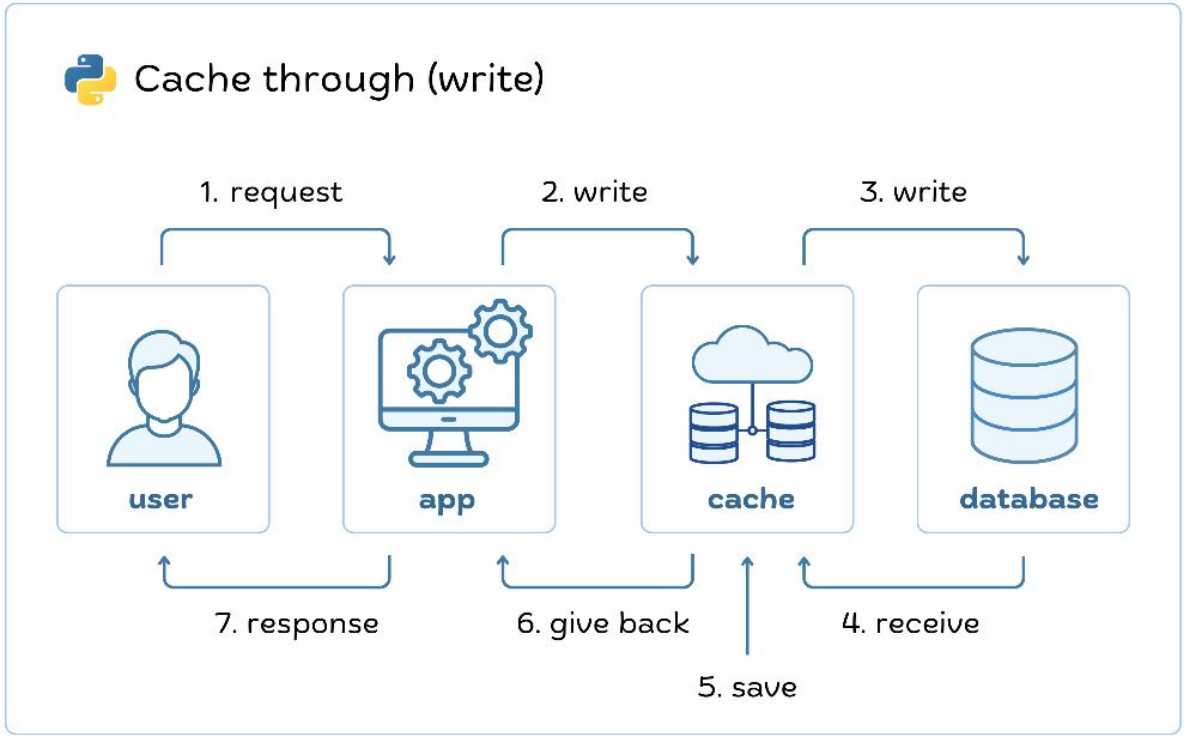
Cache Ahead
Predictor → Cache
App ← Cache

Write-behind
App → Cache ...
(Asynchronous)
DB

Вибір залежить від вимог: Cache Aside - надійність та контроль. Write-behind - максимальна швидкість запису.

Cache Through

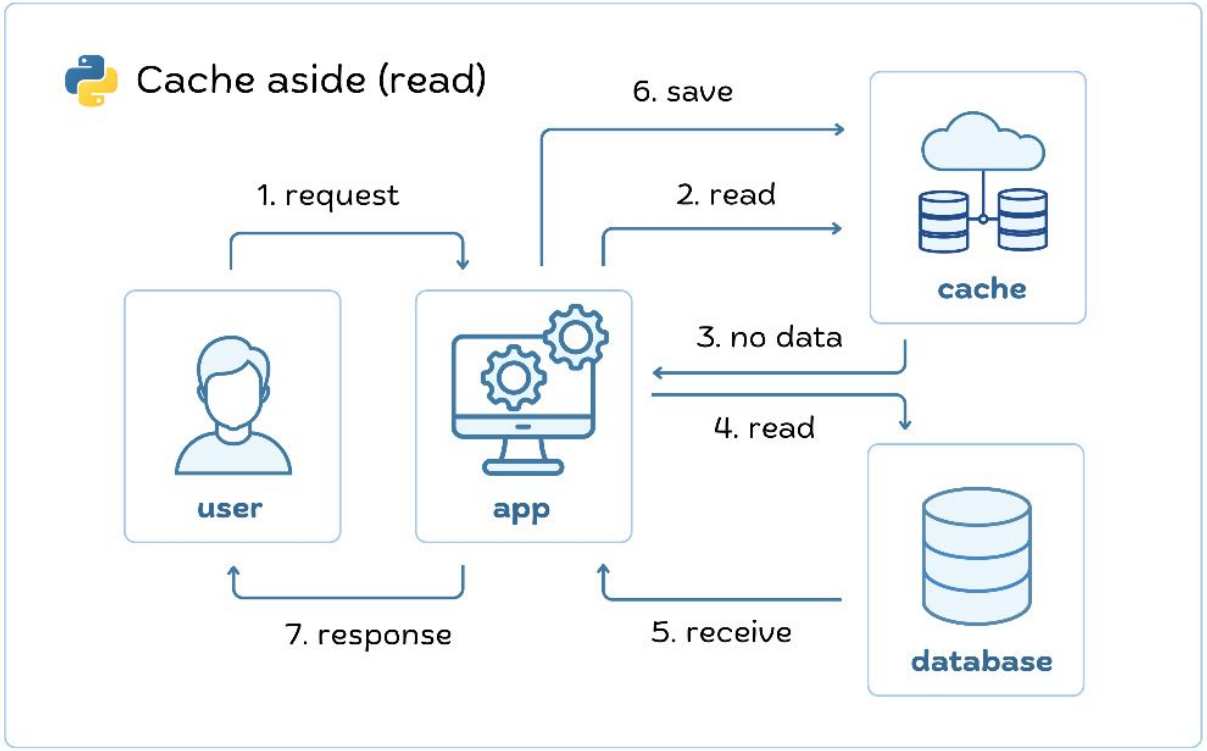


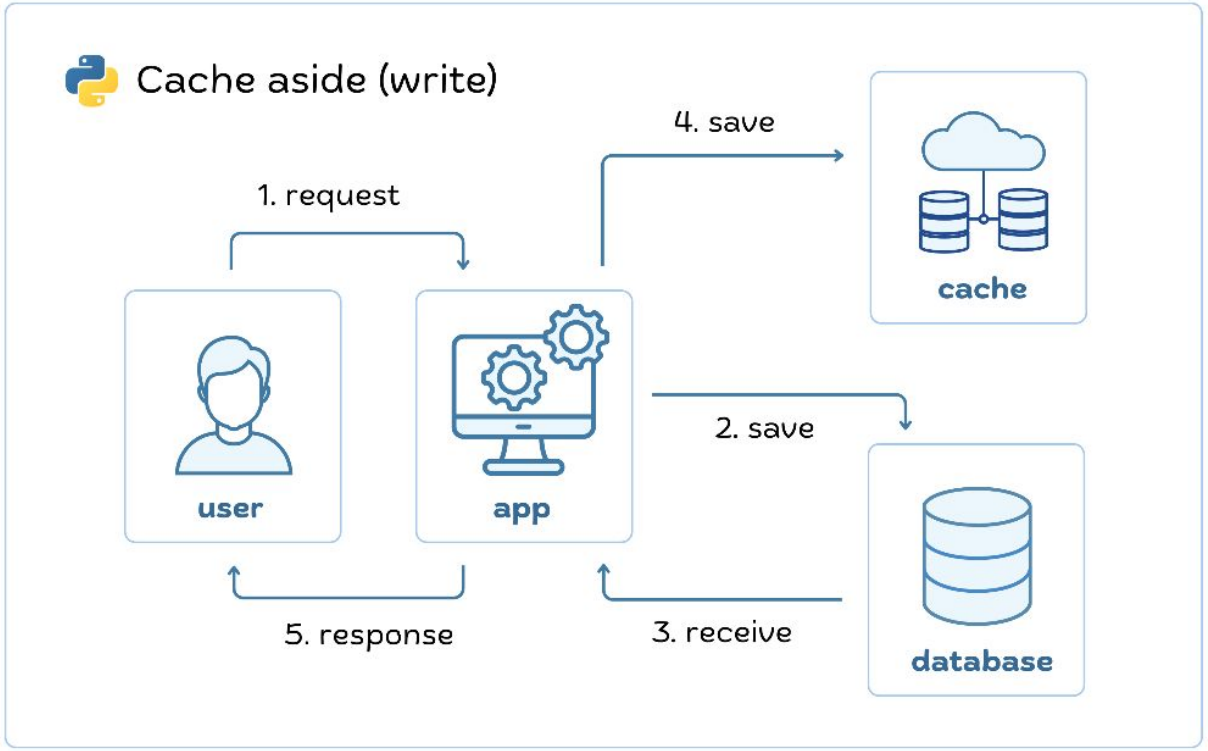


Ризик: Консистентність, валідність даних

Кеш є частиною шляху запиту: застосунок завжди працює через кеш, а не напряму з БД.

Cache Aside (Lazy Loading)

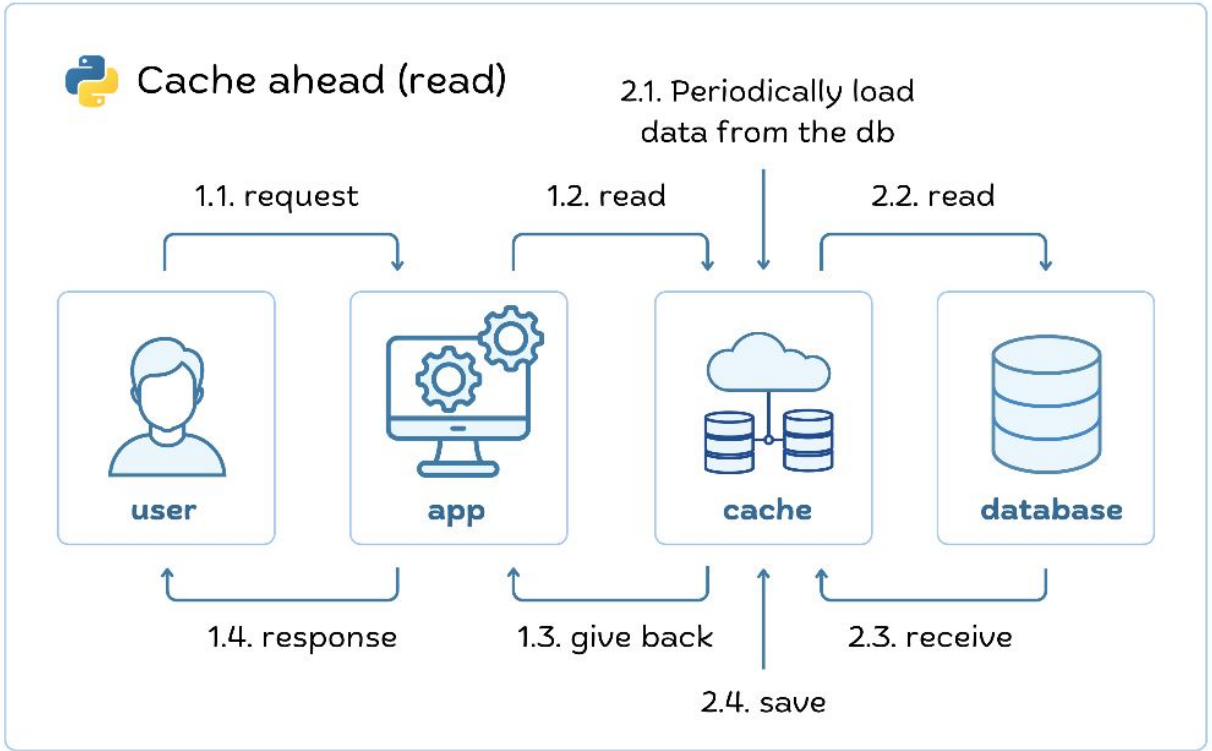


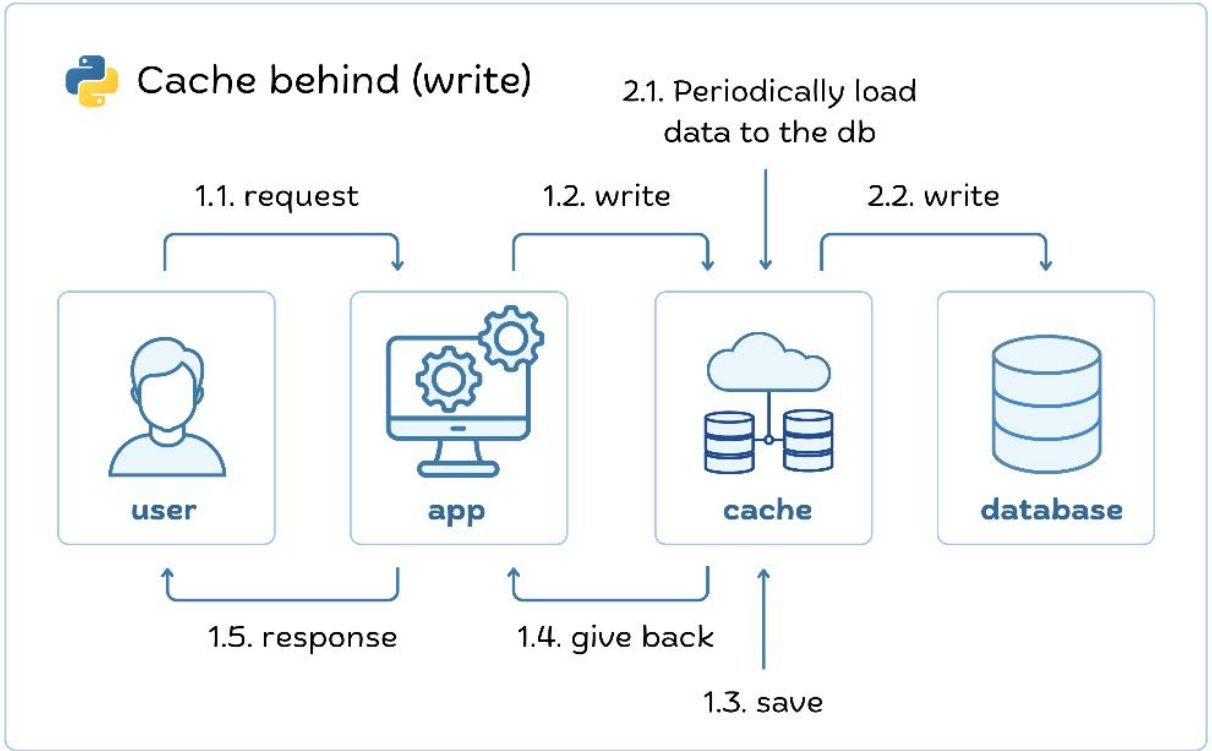


Ризик: Cache Stampede

Коли "гарячий" ключ протухає (TTL expires) або видаляється, тисячі одночасних запитів можуть пробитися до БД, створюючи лавину навантаження.

Cache Ahead vs Write-behind





Ризик: Розсинхронізація

Є ризик втратити дані при падінні кешу до синхронізації, так само як і питання тимчасової розсинхронізації

Redis vs Memcached

Memcached

Простота & Швидкість

Мультипоточність

Ефективне використання всіх ядер CPU. Ідеально для простих KV операцій під високим навантаженням.

Тільки Key-Value

Зберігає лише рядки/блоби. Жодних складних структур даних.

`Strings` `Objects (serialized)`

Керування пам'яттю

Slab allocation запобігає фрагментації. Дуже стабільне використання RAM.

КОЛИ ОБИРАТИ?

Для простого кешування об'єктів/сторінок, коли потрібно масштабування на багато ядер.

vs

Redis

Функціональність

Багаті структури даних

Дозволяє складні операції на сервері (сортування, підрахунок, гео).

`Lists` `Sets` `Hashes` `Sorted Sets` `Bitmaps`

Персистентність

RDB (снапшоти) та AOF (лог операцій). Дані не зникають після перезапуску.

Розширені можливості

Pub/Sub для повідомлень, Lua скрипти, транзакції, Redis Cluster.

КОЛИ ОБИРАТИ?

Як основний кеш, брокер повідомлень або коли потрібні складні типи даних (черги, лічильники).

Консистентність та інвалідація

> Немає безкоштовної консистентності → завжди баланс між свіжістю, швидкістю і складністю. Питання не “як зробити правильно”, а “де допустиме застаріння даних”. Консистентність - це компроміс.

TTL-only (Eventual Consistency)

Найпростіший підхід. Дані оновлюються лише після закінчення часу життя. Ризик: користувач бачить застаріле.

Інвалідація при записі

Явне видалення/оновлення ключа при зміні даних у БД. Використовуйте write-through або події.

Версіонування ключів

Зміна версії в імені ключа (user:v2:123) миттєво робить старі дані недоступними.

Захист від Stampede

Stale-while-revalidate, блокування (locking) або ймовірнісний підхід (jitter) для запобігання лавині запитів.

СПЕКТР КОНСИСТЕНТНОСТІ

Easy / Fast	Pub/Sub	Write-Through
TTL Only	Eventual	Strong

Cache Stampede (Dogpiling): Коли "гарячий" ключ зникає, сотні процесів одночасно намагаються перерахувати його, вбиваючи БД. Використовуйте Locking.

Ключові практики кешування

01

Детермінованість

Кешуйте лише "чисті" функції, результат яких залежить виключно від аргументів. Нестабільні дані (рандом, час) — ворог кешу.

02

Розумний TTL

Підбирайте час життя (TTL) під бізнес-вимоги свіжості даних. Занадто довгий TTL веде до старіння, занадто короткий — до навантаження.

03

Вимірюйте Hit Ratio

Слідкуйте за метриками. Низький Hit Ratio означає, що кеш неефективний або переповнений сміттям (cache thrashing).

04

Захист від Stampede

Використовуйте блокування (locking) або ймовірнісний підхід (probabilistic early expiration), щоб уникнути лавиноподібних запитів при протуханні ключа.

05

Прогрів та Моніторинг

Плануйте "прогрів" кешу (pre-warming) після деплою та налаштуйте алерти на використання пам'яті та помилки evictions.

Золоте правило

"Кешування — це не спосіб виправити повільний код, а інструмент для масштабування швидкого коду."

ENGINEERING WISDOM

Що почитати?

- Functools - lru_cache
- Cachetools
- AWS. Caching challenges and strategies
- Cache replacement policies
- Stanford — Dynamic Programming notes
- Caching in Python Using the LRU